

The Calkin–Wilf digit golden law

A proof of TLMC conjecture 00000000399 (binary digits)

AlyciaBHZ, on behalf of the Omega Institute

5 October 2026

Conjecture 00000000399 (verbatim).

Definition: The maximal number of digits of denominators at level n of the Calkin–Wilf tree. Conjecture: The maximum equals $n \cdot \log_2 \varphi + O(\log n)$; and the constant $\log_2 \varphi$ comes from the branching geometry of the tree. (Calkin–Wilf digit golden law)

The typographical double hyphen in this quotation renders the en dash in “Calkin–Wilf”; its wording and mathematical symbols reproduce the quoted statement.

Claim. Under the binary-digit reading, the conjecture is true. In fact, if D_n is the maximal number of binary digits of a denominator at level n , with the root at level zero, then

$$\max\{b : (a, b) \text{ is at level } n\} = F_{n+2}, \quad D_n = \lfloor \log_2 F_{n+2} \rfloor + 1,$$

and, writing $\alpha = \log_2 \varphi$,

$$0 < D_n - n\alpha \leq \alpha + 1 \quad (n \geq 0). \quad (1)$$

Here $F_0 = 0$, $F_1 = 1$, $F_{k+2} = F_k + F_{k+1}$, and $\varphi = (1 + \sqrt{5})/2$. Thus the error is $O(1)$, and in particular $O(\log n)$. The Lean theorem proves the exact maximum, exact digit count, a uniform absolute-error bound, and the literal asymptotic assertion on $\mathbb{N}$.

Submitted by AlyciaBHZ on behalf of the Omega Institute (trureturing project, <https://github.com/the-omega-institute/trureturing>).

1 Interpretation and definitions

The word “digits” alone leaves the base unspecified. The coefficient $\log_2 \varphi$ is consistent with a digit count only in base two. Theorem 6, formalized in Lean, gives the slope $\log_b \varphi$ for every integer base $b \geq 2$. Theorem 7, also formalized, shows that every $b \geq 3$ produces unbounded error against the printed coefficient, and that this error is not $O(\log n)$. Thus the conjecture is true under the binary reading and false under every other admissible integer digit base, including decimal. This conclusion follows from the all-base law rather than from a choice of base made to match the claim. A positive integer m has

$$d_2(m) = \lfloor \log_2 m \rfloor + 1$$

binary digits, with no leading zeros. The digit count of zero is immaterial: no zero denominator occurs. For definiteness, Mathlib’s digit list represents zero by an empty list, so the Lean definition gives $d_2(0) = 0$.

We use the standard Calkin–Wilf rule [1]: start with $1/1$ and give a/b the children $a/(a+b)$ and $(a+b)/b$. More precisely, put

$$L_0 = \{(1, 1)\}, \quad L_{n+1} = \{(a, a+b) : (a, b) \in L_n\} \cup \{(a+b, b) : (a, b) \in L_n\}.$$

The finite set representation records all level labels; multiplicities cannot affect a maximum. Define $M_n = \max_{(a,b) \in L_n} b$ and $D_n = \max_{(a,b) \in L_n} d_2(b)$. Each level is nonempty, as follows also from the explicit witnesses below.

All coordinates are positive by induction. Moreover,

$$\gcd(a, a+b) = \gcd(a, b) = \gcd(a+b, b).$$

Since the root is coprime, every generated fraction is already in lowest terms. Consequently these second coordinates are the actual reduced fraction denominators; no unmentioned reduction changes the maximization. This property is separately proved in Lean as `level_wellformed`.

The phrase “comes from the branching geometry” is explanatory rather than an additional quantitative assertion with a formal definition. Our proof exhibits its mechanism: a sharp bound on coordinate sums survives the two branching operations, and alternating branches attain successive Fibonacci labels. The characteristic growth equation is $x^2 = x + 1$, with positive root φ .

2 The exact denominator maximum

Lemma 1 (Sharp coordinate and sum invariant). *For every $n \geq 0$ and every $(a, b) \in L_n$,*

$$a \leq F_{n+2}, \quad b \leq F_{n+2}, \quad a+b \leq F_{n+3}.$$

Proof. At $n = 0$ the coordinates are $1 = F_2$, and their sum is $2 = F_3$. Assume the invariant at level n . At the left child $(a, a+b)$, the coordinates are bounded by F_{n+3} , since $F_{n+2} \leq F_{n+3}$, and the sum is bounded by

$$a + (a+b) \leq F_{n+2} + F_{n+3} = F_{n+4}.$$

At the right child $(a+b, b)$ the same coordinate bounds apply, and

$$(a+b) + b \leq F_{n+3} + F_{n+2} = F_{n+4}.$$

These are exactly the three assertions at level $n+1$. □

Lemma 2 (Attained Fibonacci labels). *For every $n \geq 0$, both*

$$(F_{n+1}, F_{n+2}) \quad \text{and} \quad (F_{n+2}, F_{n+1})$$

belong to L_n .

Proof. At level zero both labels are $(1, 1)$. The left child of (F_{n+2}, F_{n+1}) is $(F_{n+2}, F_{n+2}+F_{n+1}) = (F_{n+2}, F_{n+3})$. The right child of (F_{n+1}, F_{n+2}) is (F_{n+3}, F_{n+2}) . This proves both assertions simultaneously by induction. Tracing either family backward alternates the left and right operations; the two families are mirror zigzags. □

Theorem 3 (Exact maxima). *For all $n \geq 0$, $M_n = F_{n+2}$ and $D_n = d_2(F_{n+2}) = \lfloor \log_2 F_{n+2} \rfloor + 1$.*

Proof. Lemma 1 bounds every denominator by F_{n+2} , and the first witness in Lemma 2 attains it. Digit count is monotone in a positive integer: increasing the integer cannot decrease its logarithm or the floor of that logarithm. Hence the same witness attains the maximum digit count. Since $F_{n+2} > 0$, the usual floor formula applies. $\square$

For instance, $M_0, M_1, M_2, M_3 = 1, 2, 3, 5$, and the corresponding maximum binary digit counts are 1, 2, 2, 3.

3 Growth and the digit error

Lemma 4. *For all $n \geq 0$,*

$$\varphi^n \leq F_{n+2} \leq \varphi^{n+1}. \quad (2)$$

Proof. We have $1 < \varphi < 2$ and $\varphi^2 = \varphi + 1$. At $n = 0$, $1 \leq F_2 = 1 \leq \varphi$. At $n = 1$, $\varphi \leq F_3 = 2 \leq \varphi^2$. If (2) holds at n and $n + 1$, add the respective bounds and use the Fibonacci recurrence:

$$\varphi^n + \varphi^{n+1} \leq F_{n+4} \leq \varphi^{n+1} + \varphi^{n+2}.$$

The outside expressions equal φ^{n+2} and φ^{n+3} , respectively, by $\varphi^2 = \varphi + 1$. This is (2) at $n + 2$. Induction with two initial cases proves the lemma. $\square$

Theorem 5 (Uniform error and the conjectured asymptotic). *For every $n \geq 0$, (1) holds. In particular,*

$$D_n = n \log_2 \varphi + O(1) = n \log_2 \varphi + O(\log n) \quad (n \rightarrow \infty).$$

Proof. Taking base-two logarithms in (2) gives

$$n\alpha \leq \log_2 F_{n+2} \leq (n+1)\alpha.$$

For any positive integer m ,

$$\log_2 m < d_2(m) \leq \log_2 m + 1.$$

Apply this to $m = F_{n+2}$ and use Theorem 3. The resulting inequalities are precisely (1). Thus $|D_n - n\alpha| \leq \alpha + 1$ at every level, giving $O(1)$. Finally, $\log n \rightarrow +\infty$ as $n \rightarrow \infty$ on the natural numbers, so eventually $\log n \geq 1$ and

$$|D_n - n\alpha| \leq \alpha + 1 \leq (\alpha + 1)|\log n|.$$

This is the defining norm estimate for the literal $O(\log n)$ assertion. The value $\log 1 = 0$ causes no issue: big-O at infinity concerns an eventual bound, whereas the separate uniform constant bound holds for all n . $\square$

Theorem 6 (Every integer digit base). *Fix an integer $b \geq 2$. Define $d_b(m) = \lfloor \log_b m \rfloor + 1$ for $m > 0$ and $D_n^{(b)} = \max_{(a,m) \in L_n} d_b(m)$. Then for all $n \geq 0$,*

$$D_n^{(b)} = \lfloor \log_b F_{n+2} \rfloor + 1, \quad 0 < D_n^{(b)} - n \log_b \varphi \leq \log_b \varphi + 1.$$

In particular, $D_n^{(b)} = n \log_b \varphi + O(1)$ with explicit absolute-error bound $\log_b \varphi + 1$.

Proof. The attained denominator maximum is independent of the digit base, and d_b is monotone. Taking base- b logarithms in (2) gives $n \log_b \varphi \leq \log_b F_{n+2} \leq (n+1) \log_b \varphi$. Combine this with $\log_b m < d_b(m) \leq \log_b m + 1$. $\square$

Theorem 7 (The binary coefficient is unique). *For every integer $b \geq 3$, put $\delta_b = \log_2 \varphi - \log_b \varphi > 0$. Then for every $n \geq 0$,*

$$\left| D_n^{(b)} - n \log_2 \varphi \right| \geq n \delta_b - (\log_b \varphi + 1).$$

This error is unbounded and is not $O(\log n)$. Among integer bases $b \geq 2$, the conjecture's $O(\log n)$ assertion holds if and only if $b = 2$.

Proof. Since $\log \varphi > 0$ and $\log b > \log 2 > 0$, the quotient $\log \varphi / \log b$ is strictly less than $\log \varphi / \log 2$. Set $e_n = D_n^{(b)} - n \log_b \varphi$, so that $0 < e_n \leq \log_b \varphi + 1$ by Theorem 6. The absolute error against the binary slope is $|e_n - n \delta_b| \geq n \delta_b - e_n$, proving the lower bound. For any real C , choose a natural $n > (C + \log_b \varphi + 1) / \delta_b$ to obtain error greater than C .

Unboundedness alone does not rule out $O(\log n)$. For that stronger conclusion, use $\log n = o(n)$. If the error were $O(\log n)$, it would be $o(n)$ and eventually at most $(\delta_b/2)n$. The lower bound exceeds that quantity when $n > 2(\log_b \varphi + 1) / \delta_b$, a contradiction. The binary case is Theorem 5. $\square$

4 Lean statement and semantic correspondence

The project pins Lean v4.33.0 and Mathlib v4.33.0 (commit below).

db584cd6d46c92f209a44c0f1c829460d327499d

Its sole source file is `lean4/GoldenLaw.lean`, in namespace `CalkinWilf399`. Its definitions are:

```
def left (p : Nat x Nat) := (p.1, p.1 + p.2)
def right (p : Nat x Nat) := (p.1 + p.2, p.2)
level 0 = {(1, 1)}
level (n + 1) = (level n).image left union (level n).image right
maxDen n = (level n).sup Prod.snd
digits2 m = (Nat.digits 2 m).length
maxDigits n = (level n).sup (fun p => digits2 p.2)
digitsBase b m = (Nat.digits b m).length
maxDigitsBase b n = (level n).sup (fun p => digitsBase b p.2)
```

Here the ASCII `x` and `union` display product and union notation; the definitions in the source use Lean's actual Unicode notation. The finite supremum in $\mathbb{N}$ is a maximum on a nonempty finite set. The witness theorem establishes nonemptiness and attainment without an assumption about the computed maximum.

For readability, let α stand for the exact Lean term `Real.logb 2 Real.goldenRatio`. The main theorem is `CalkinWilf399.calkin_wilf_digit_golden_law`; its statement, with that one abbreviation and ASCII display notation, is:

```
(\forall n, maxDen n = Nat.fib (n+2)) /\
(\forall n, maxDigits n = digits2 (Nat.fib (n+2))) /\
(\exists C : Real, \forall n : Nat, 1 <= n ->
  |(maxDigits n : Real) - (n : Real) * alpha| <= C) /\
((fun n : Nat => (maxDigits n : Real) - (n : Real) * alpha)
 =0[atTop] (fun n : Nat => Real.log (n : Real)))
```

The displayed `\forall`, `\exists`, `/`, and `Real` are textual equivalents of Lean’s quantifiers, conjunction, and real-number type. No mathematical hypothesis is suppressed. The literal compilable statement is in the source.

`level_bounds` proves Lemma 1, and `fibonacci_witnesses` proves Lemma 2. `maxDen_eq_fib` and `maxDigits_eq_fib_digits` prove the maxima over the recursively defined level, rather than defining the maximum to be a Fibonacci number. `fib_golden_bounds` proves Lemma 4. `digits2_log_bounds` uses Mathlib’s digit-length and logarithm-floor identities. `bounded_error` proves the explicit absolute bound with $C = \log_2 \varphi + 1$ for every n , including zero.

`maxDigits_eq_size` also identifies the maximum with `Nat.size (Nat.fib (n+2))`; `maxDigits_eq_log` gives `Nat.log 2 (Nat.fib (n+2)) + 1`.

`exists_bounded_error` gives the existential form for $n \geq 1$, and `digit_error_isBig0_log` proves the eventual norm estimate in Mathlib’s `Asymptotics.IsBig0`.

The all-base definitions use the same recursively generated levels. `maxDigitsBase_eq_log` proves the exact integer-log formula for $b \geq 2$. `base_digit_error_bounds` and `base_bounded_error` prove Theorem 6. `binary_coefficient_error_lower` proves the explicit linear lower bound in Theorem 7; the unboundedness theorem is `binary_coefficient_error_unbounded`. `binary_coefficient_error_not_isBig0_log` proves the actual negation of Mathlib’s asymptotic proposition, and `binary_coefficient_valid_iff` proves its equivalence to $b = 2$. The existing binary definitions and theorems are retained; `maxDigitsBase_two` identifies the base-two specialization.

Axiom audit. The source ends with `#print axioms` for the original binary main theorem and ten supporting or extended results. The successful compiler output in `verification.txt` shows only

```
[propext, Classical.choice, Quot.sound]
```

for each audited theorem. These are Lean’s standard axioms. The project adds no axioms and contains no incomplete proofs or kernel-bypass tactics. No `trurerturing` code is vendored; the proof is original to this package and uses Mathlib lemmas directly.

5 Other readings and historical context

In particular, Theorems 6 and 7 give

$$D_n^{(10)} = \lfloor \log_{10} F_{n+2} \rfloor + 1 = n \log_{10} \varphi + O(1),$$

with explicit bound $\log_{10} \varphi + 1$. Its error against $n \log_2 \varphi$ is unbounded and not $O(\log n)$. These decimal and general-base claims are consequences of the accompanying Lean theorems, rather than additional observations confined to the report. Changing the convention to put the root at level one replaces F_{n+2} by F_{n+1} and shifts D_n by one level. This changes only the bounded error, so the claimed asymptotic coefficient remains valid.

For context, Stern’s diatomic sequence is defined by $s(0) = 0$, $s(1) = 1$, $s(2k) = s(k)$, and $s(2k + 1) = s(k) + s(k + 1)$. The breadth-first labels at level n are

$$\frac{s(j)}{s(j+1)}, \quad 2^n \leq j < 2^{n+1}.$$

This follows inductively because the two children at indices $2j, 2j + 1$ have labels $(s(j), s(j) + s(j + 1))$ and $(s(j) + s(j + 1), s(j + 1))$. Consequently our denominator theorem also yields

$$\max_{2^n < k \leq 2^{n+1}} s(k) = F_{n+2}.$$

The often-used interval $[2^n, 2^{n+1}]$ has the same maximum, because its additional left endpoint has value $s(2^n) = 1$. This discussion is a bridge to classical diatomic-sequence literature, not a lemma assumed by the Lean proof. Calkin and Wilf [1] explain the rational enumeration, and Lehmer [2] is a classical reference on Stern’s diatomic series. Lucas’s 1878 work [3] supplies historical context for recurrence sequences. We do not claim that the present digit statement, or the precise interval maximum with our indexing, is a theorem quoted from Lucas. The tree argument above independently proves the exact bound needed here.

6 Reproduction and supplementary computation

From the `lean4/` directory in a normal Lean installation, run

```
lake exe cache get && lake build
```

Compilation against the pinned Mathlib environment succeeded. The recorded compiler output, including the standard-axiom audit and successful exit code, is preserved in `verification.txt`.

Run `python3 verify.py` for the independent standard-library check. It generates the actual binary tree through level 17, checks positivity, coprimality, the three inequalities, both witnesses, and the two maxima. It also computes Stern’s sequence independently and checks every label against the breadth-first enumeration. Finally, it checks the explicit error bound numerically through level 10000 using exact integer Fibonacci numbers and exact digit lengths in bases 2, 3, 4, 10, 16. It checks the all-base bounds and the linear lower bound against the binary coefficient. The floating-point error checks are only sanity checks: every assertion for all levels is proved in Lean without numerical approximations.

Build this report with `pdflatex report.tex` twice. The delivered PDF and TeX source are self-contained.

References

- [1] N. Calkin and H. S. Wilf, Recounting the rationals, *American Mathematical Monthly* 107 (2000), no. 4, 360–363.
- [2] D. H. Lehmer, On Stern’s diatomic series, *American Mathematical Monthly* 36 (1929), no. 2, 59–67.
- [3] E. Lucas, Théorie des fonctions numériques simplement périodiques, *American Journal of Mathematics* 1 (1878), 184–240 and 289–321.